

ARYABHATTA KNOWLEDGE UNIVERSITY, PATNA

BCA — SEMESTER III EXAMINATION

OBJECT-ORIENTED PROGRAMMING USING C++

Paper Code	BCA-301 / 303301	Full Marks	60
Time	3 Hours	Attempt	5 out of 7 (Q1 & Q2 compulsory)
PYQ Years	2014, 2017, 2018, 2019, 2020, 2021, 2022, 2024	Text Books	E. Balagursamy — OOP with C++ (TMH)

CONTENTS TABLE

Unit	Topic	MCQs	4-Mark	12-Mark
1	Introduction to Programming Paradigms	5	3	2
2	Basics of C++	5	3	2
3	Operator Overloading	5	3	2
4	Inheritance	5	3	2
5	Advanced Topics (Templates, Exception, File)	5	3	2
—	Most Repeated Questions + Cheat Sheet	—	—	—

VVI MOST IMPORTANT TOPICS

- ★★★★★ Constructor & Destructor — Asked EVERY year (2014–2024)
- ★★★★★ Types of Inheritance — 6+ times (2014,2017,2018,2021,2022,2024)
- ★★★★★ OOP Features/Paradigm — Asked EVERY year (2014–2024)
- ★★★★★ Virtual Functions & Dynamic Binding — 6 times
- ★★★★★ Operator Overloading — 5+ times
- ★★★★★ Templates (Function + Class) — 5 times
- ★★★★★ Exception Handling — 5 times
- ★★★ Abstract Class + Pure Virtual Function — 5 times

UNIT 1 — INTRODUCTION TO PROGRAMMING PARADIGMS

Class & Object · OOP Characteristics · Data Hiding · Encapsulation · Data Security · Class vs Struct

SECTION A — MCQ QUESTIONS (2 Marks Each)

MCQ 1. Which access specifier is used in a class by default? [2014, 2019]

- (a) Public
- (b) Protected
- (c) **Private ✓**
- (d) Friend

Explanation: Class members are *PRIVATE* by default. Struct members are *PUBLIC*. Key difference between class and struct.

MCQ 2. The OOP characteristic that hides internal data is: [2018]

- (a) Abstraction
- (b) **Encapsulation ✓**
- (c) Inheritance
- (d) Polymorphism

Explanation: *ENCAPSULATION* hides data by binding data+methods. Private members make data inaccessible from outside.

MCQ 3. An object is an ____ of a class. [2020]

- (a) Definition
- (b) Function
- (c) **Instance ✓**
- (d) Structure

Explanation: Object is an *INSTANCE* of a class. Class is the blueprint; object is the actual entity created from it.

MCQ 4. By default, members of a struct are: [2018]

- (a) Private
- (b) Protected
- (c) **Public ✓**
- (d) None

Explanation: *STRUCT* members are *PUBLIC* by default. *CLASS* members are *PRIVATE*. Primary syntax difference.

MCQ 5. Data hiding is achieved by declaring members as: [2014]

- (a) public
- (b) protected
- (c) static
- (d) **private ✓**

Explanation: *PRIVATE* members cannot be accessed from outside — only through public methods. This is data hiding.

SECTION B — SHORT ANSWER QUESTIONS (4 Marks Each)

Q. Explain the characteristics of OOP.

★ PYQ: ★ 2017,2018,2019

OOP organizes programs around objects.

- Encapsulation: Binding data + methods in one unit. Hides internal details.
- Data Hiding: Private members inaccessible from outside.
- Data Security: Prevents unauthorized access via access specifiers.
- Abstraction: Show only what is needed; hide complexity.

- Inheritance: Derived class gets properties of base class.
- Polymorphism: Same name, different behavior.

Q. What is the difference between a class and a structure?

★ PYQ: 2018, 2020

Key syntactic differences between class and struct in C++.

- Default Access: Class = private | Struct = public
- Default Inheritance: Class = private | Struct = public
- Usage: Class for full OOP; Struct for simple data grouping
- Keyword: 'class' vs 'struct'
- Code: struct S { int x; }; → x is public | class C { int x; }; → x is private

Q. What are private and public members of a class?

★ PYQ: 2014

Access specifiers control member visibility.

- Private Members: Accessible ONLY within the same class. Cannot be accessed from outside.
- Public Members: Accessible from ANYWHERE in the program.
- Protected Members: Accessible within class and in derived classes.
- Default in class: private | Default in struct: public

SECTION C — LONG ANSWER QUESTIONS (12 Marks Each)

Q. Explain the OOP paradigm with all its essential features.

★ PYQ: ★★★★★ EVERY YEAR: 2014–2024

Introduction:

OOP organizes software around objects rather than actions/functions. C++ fully supports OOP.

1. Class & Object:

Class is blueprint/template. Object is an instance with its own data and methods.

2. Encapsulation:

Binding data + methods in one unit (class). Access specifiers control visibility. Promotes data security.

3. Data Abstraction:

Hiding internal implementation. User sees only interface. E.g., use cout without knowing its code.

4. Data Hiding:

Private members inaccessible from outside. Protects data integrity and prevents unauthorized modification.

5. Inheritance:

Derived class acquires properties of base class. Promotes code reusability.

6. Polymorphism:

Same name, different behavior. Compile-time: overloading. Runtime: virtual functions.

7. Message Passing:

Objects communicate by calling member functions.

```
#include
using namespace std;
class Student {
private:
    int roll; string name;
public:
    Student(int r, string n){roll=r;name=n;}
    void display(){cout<<};
int main(){
    Student s1(101, "Sana");
```

```
s1.display();  
return 0;  
}
```

Conclusion: OOP makes software more manageable, reusable, and secure.

Q. Explain the concept of class and object. How does object differ from a variable?

★ PYQ: PYQ 2018, 2020

Introduction:

Class and Object are fundamental building blocks of C++ OOP.

Class:

User-defined data type grouping data members + member functions. Blueprint for objects.

Object:

Instance of a class. Memory allocated when object created, not when class is defined.

Object vs Variable:

Variable: stores simple data | Object: stores data AND functions. Variable: predefined type | Object: user-defined type. Object supports encapsulation; variable does not.

```
class Car{  
public:  
    string model;  
    int speed;  
    void show(){cout<<};  
int main(){  
    Car c1;  
    c1.model="Swift";  
    c1.speed=120;  
    c1.show();  
}
```

Conclusion: Class defines structure; object is the real entity. Objects are variables of class type with extra capabilities.

UNIT 2 — BASICS OF C++

Structure of C++ Programs · Constructors · Destructors · cin/cout · Friend Functions · this Pointer · new/delete · Static Members

SECTION A — MCQ QUESTIONS (2 Marks Each)

MCQ 1. cout stands for: [2018]

- (a) Class Output
- (b) Character Output
- (c) **Console Output ✓**
- (d) Call Output

Explanation: cout = Console OUTput. Object of ostream class. Similarly cin = Console INput.

MCQ 2. If no destructor provided, ____ creates an empty one. [2014]

- (a) Linker
- (b) **Compiler ✓**
- (c) Preprocessor
- (d) OS

Explanation: COMPILER automatically generates default constructor/destructor if not explicitly defined.

MCQ 3. Which of the following is NOT a member of a class? [2014]

- (a) Static function
- (b) **Friend function ✓**
- (c) Const function
- (d) Virtual function

Explanation: FRIEND function is declared inside class but NOT a class member. Can access private data.

MCQ 4. Which cannot be declared static? [2018]

- (a) **Class ✓**
- (b) Object
- (c) Functions
- (d) Data members

Explanation: A CLASS itself cannot be declared static. Objects, data members, member functions can be static.

MCQ 5. The 'this' pointer refers to: [2022]

- (a) Base class
- (b) Derived class
- (c) **Current object ✓**
- (d) Static object

Explanation: 'this' is a constant pointer holding address of the CURRENT OBJECT. Available in non-static member functions.

SECTION B — SHORT ANSWER QUESTIONS (4 Marks Each)

Q. What is a constructor? Explain its characteristics.

★ PYQ: ★★★★★ 2017,2018,2019,2020,2021

Constructor is auto-called when object is created.

- Same name as the class.
- No return type (not even void).
- Called automatically when object is created.
- Can be overloaded (multiple constructors allowed).

- Cannot be declared virtual.
- Types: Default (no params), Parameterized (with args), Copy (from same class object).

Q. What is a destructor? How does it differ from a constructor?

★ PYQ: 2014,2018,2022,2024

Destructor is auto-called when object is destroyed.

- Name = tilde (~) + class name: ~ClassName()
- No return type, no parameters.
- Cannot be overloaded — only ONE destructor per class.
- Called in REVERSE order of constructors.
- vs Constructor: Constructor=creation | Destructor=destruction
- vs Constructor: Constructor can be overloaded | Destructor cannot

Q. What is a friend function? Why is it needed?

★ PYQ: ★★★ 2014,2019,2021,2022,2024

Friend function accesses private data without being a member.

- NOT a member of the class but declared inside with 'friend' keyword.
- Can access private and protected members.
- No 'this' pointer — not called on object.
- Used for operator overloading (especially << and >>).
- Allows controlled access without making data public.

SECTION C — LONG ANSWER QUESTIONS (12 Marks Each)

Q. Write a C++ program demonstrating all types of constructors and destructor.

★ PYQ: ★★★★★ 2017,2018,2019,2020,2021,2024

Introduction:

C++ provides 3 constructor types: Default, Parameterized, and Copy. Destructor performs cleanup.

Default Constructor:

No parameters. Auto-called when object created without args.

Parameterized Constructor:

Takes arguments to initialize object with specific values.

Copy Constructor:

Creates new object as copy of existing. Must use reference parameter.

Destructor:

Auto-called when object destroyed. Releases allocated resources.

```
#include
using namespace std;
class Student{
    int roll; float marks;
public:
    Student(){roll=0;marks=0.0; cout<<"Default\n";}
    Student(int r,float m){roll=r;marks=m; cout<<"Parameterized r="<    Student(const Student &s;){roll=s.
roll;marks=s.marks; cout<<"Copy\n";}
    void display(){cout<<"Roll:"<    ~Student(){cout<<"Destructor for "<};
int main(){
    Student s1;                // Default
    Student s2(101,95.5f);    // Parameterized
    Student s3(s2);            // Copy
    s2.display(); s3.display();
    return 0; // Destructors: s3,s2,s1 (reverse)
}
```

Conclusion: Constructors initialize objects automatically. Destructor cleans up resources when object is destroyed.

Q. Explain friend function. Write a program showing use of friend functions.

★ PYQ: ★★ ★ 2019,2021,2022,2024

Introduction:

Friend function breaks encapsulation barrier for selective access.

Characteristics:

Declared inside class with 'friend' keyword. Defined outside. NOT a member. No 'this' pointer.

When to Use:

Access private data of two classes. Operator overloading. Selective controlled access.

```
#include
using namespace std;
class Rectangle{
    float length,width;
public:
    Rectangle(float l,float w){length=l;width=w;}
    friend float area(Rectangle r);
    friend ostream& operator<<(ostream& out,Rectangle r);
};
float area(Rectangle r){
    return r.length*r.width; // Access private!
}
ostream& operator<<(ostream& out,Rectangle r){
    out<<    return out;
}
int main(){
    Rectangle r1(5.0,3.0);
    cout<<"Size: "<<    cout<<"Area: "<<    return 0;
}
```

Conclusion: Friend functions enable controlled access to private data. Essential for operator overloading.

UNIT 3 — OPERATOR OVERLOADING

Operator Functions · Binary/Unary Operators · Friend Functions for Operators · Stream Operators · Type Conversion

SECTION A — MCQ QUESTIONS (2 Marks Each)

MCQ 1. Use of the function call operator (): [2014]

- (a) Overloading method
- (b) **Overloading objects ✓**
- (c) Overloading parameters
- (d) None

Explanation: Function call () operator allows objects to be used like callable functions (functors).

MCQ 2. Arguments for binary operator as MEMBER function: [2017]

- (a) 0
- (b) **1 ✓**
- (c) 2
- (d) 3

Explanation: As MEMBER: binary=1 arg ('this'=left operand). As FRIEND: binary=2 args.

MCQ 3. Which operator CANNOT be overloaded? [Not verified]

- (a) + (plus)
- (b) == (equal)
- (c) **:: (scope resolution) ✓**
- (d) [] (subscript)

Explanation: Cannot overload: :: . * ?: sizeof. Scope resolution, member access, ternary CANNOT be overloaded.

MCQ 4. For overloading << and >>, we must use: [2014,2017,2018]

- (a) Member function
- (b) Static function
- (c) **Friend function ✓**
- (d) Virtual function

Explanation: Stream operators MUST be friend functions (left operand is cin/cout). Must return stream reference.

MCQ 5. Hiding complexity from users is called: [2018]

- (a) **Abstraction ✓**
- (b) Encapsulation
- (c) Data hiding
- (d) Composition

Explanation: ABSTRACTION=hiding complexity. ENCAPSULATION=binding data+methods. DATA HIDING=restricting access.

SECTION B — SHORT ANSWER QUESTIONS (4 Marks Each)

Q. What is an operator function? Describe its syntax.

★ PYQ: 2014, 2022

Operator function defines behavior of operator for user-defined types.

- Uses 'operator' keyword followed by the operator symbol.
- As member function: return_type operator op(argument) { }
- As friend function: friend return_type operator op(arg1,arg2);
- Example: Complex operator+(const Complex &c){ return Complex(real+c.real,imag+c.imag); }

Q. Write syntax for overloading >> and << operators with friend function.

★ PYQ: ★★★ 2014,2017,2018

Stream operators MUST be friends returning stream reference.

- Insertion (<<): friend ostream& operator<<(ostream &out,const Obj &o){ out<<data; return out; }
- Extraction (>>): friend istream& operator>>(istream &in,Obj &o){ in>>data; return in; }
- MUST be friend (left operand is cin/cout, not class object).
- MUST return stream reference (for chaining: cout<<a<<b).

Q. What is a copy constructor? When is it called?

★ PYQ: 2017,2019,2021,2024

Copy constructor creates new object as copy of existing object.

- Syntax: ClassName(const ClassName &obj){ copy members; }
- Parameter MUST be by REFERENCE — else infinite recursion!
- Called when: (1) Student s2=s1; (2) Passed by value to function. (3) Returned by value.

SECTION C — LONG ANSWER QUESTIONS (12 Marks Each)

Q. What is operator overloading? Explain type conversion basic-to-class and class-to-basic.

★ PYQ: ★★★★★ 2014, 2018, 2022

Introduction:

Operator overloading gives special meaning to existing operators for user-defined types.

Restrictions:

Cannot create new operators. Cannot change precedence. Cannot overload :: . .* ?: sizeof.

Conversion 1: Basic→Class:

Use single-argument constructor for implicit conversion: Distance d=5; (int auto-converts)

Conversion 2: Class→Basic:

Use conversion function: operator int(){ return meters; } — Distance auto-converts to int.

```
// Basic->Class:
class Distance{ int m;
public:
    Distance(int x){m=x;} // Conversion constructor
    void show(){cout<<};
// Class->Basic:
class Dist2{ int m;
public:
    Dist2(int x){m=x;}
    operator int(){return m;} // Conversion function
};
int main(){
    Distance d=5; // int->Distance
    d.show();     // 5m
    Dist2 d2(10);
    int x=d2;     // Dist2->int
    cout<<}
```

Conclusion: Operator overloading makes code intuitive. Type conversion enables interaction between built-in and user-defined types.

Q. Write a program to overload +, - and * for Complex numbers.

★ PYQ: PYQ 2018, 2022

Introduction:

Overloading arithmetic operators for complex numbers makes code natural and readable.

Approach:

Define operator+, operator-, operator* as member functions of Complex class.

```
#include
using namespace std;
class Complex{
    float real,imag;
public:
    Complex(){real=imag=0;}
    Complex(float r,float i){real=r;imag=i;}
    Complex operator+(Complex c){return Complex(real+c.real,imag+c.imag);}
    Complex operator-(Complex c){return Complex(real-c.real,imag-c.imag);}
    Complex operator*(Complex c){
        return Complex(real*c.real-imag*c.imag,real*c.imag+imag*c.real);
    }
    void display(){cout<<};
}

int main(){
    Complex c1(3.0,4.0),c2(1.5,2.5);
    (c1+c2).display(); (c1-c2).display(); (c1*c2).display();
    return 0;
}
```

Conclusion: Overloaded operators make complex number arithmetic as natural as built-in type arithmetic.

UNIT 4 — INHERITANCE

Base & Derived Classes · Protected Members · Constructor/Destructor Order · Public/Private/Protected Inheritance · Abstract Classes · Virtual Functions · Dynamic Binding

SECTION A — MCQ QUESTIONS (2 Marks Each)

MCQ 1. Base class pointer CANNOT point to derived class. True/False? [2014]

- (a) True
- (b) False ✓

Explanation: FALSE — Base class pointer CAN point to derived. Foundation of runtime polymorphism.

MCQ 2. Mechanism by which object acquires properties of another? [2018]

- (a) Encapsulation
- (b) Abstraction
- (c) Inheritance ✓
- (d) Polymorphism

Explanation: INHERITANCE — derived class acquires data and functions of base class. Key OOP mechanism.

MCQ 3. What does derived class NOT inherit? [2014]

- (a) Friends
- (b) Operator=() members
- (c) Constructors & destructors
- (d) All of the above ✓

Explanation: Derived class does NOT inherit: friends, operator=(), and constructors/destructors.

MCQ 4. To defer implementation to derived classes, use: [2018]

- (a) Interface
- (b) Abstract class ✓
- (c) Friend class
- (d) Static class

Explanation: ABSTRACT CLASS uses pure virtual functions (=0) to force derived classes to implement them.

MCQ 5. Virtual function enables: [2017,2019]

- (a) Compile-time polymorphism
- (b) Runtime polymorphism ✓
- (c) Static binding
- (d) Data hiding

Explanation: Virtual function enables RUNTIME POLYMORPHISM. Correct derived function called via base pointer.

SECTION B — SHORT ANSWER QUESTIONS (4 Marks Each)

Q. What is abstract class? What is its benefit?

★ PYQ: ★★★★★ 2017,2018,2019,2022,2024

Abstract class contains pure virtual function and cannot be instantiated.

- Contains at least one pure virtual function: virtual void f() = 0;
- Objects of abstract class CANNOT be created directly.
- Forces derived classes to implement all pure virtual functions.
- Provides common interface for all derived classes.
- Example: Shape class with pure virtual area() forces Circle/Rectangle to implement area().

Q. What is virtual function? Why is it needed?

★ PYQ: ★★★ 2017,2019,2020,2021,2022

Virtual function enables runtime polymorphism through base class pointer.

- Declared with 'virtual' keyword in base class.
- Without virtual: base pointer always calls BASE function (static binding).
- With virtual: correct DERIVED function called at runtime (dynamic binding).
- Cannot be static. Must be member of class.
- Always use virtual destructor when using inheritance!

Q. Explain visibility of base class members with access specifiers in derived class.

★ PYQ: 2018, 2022

Inheritance visibility depends on access specifier used.

- Public inheritance: public→public, protected→protected, private→inaccessible
- Private inheritance: public→private, protected→private, private→inaccessible
- Protected inheritance: public→protected, protected→protected, private→inaccessible
- Private members of base class are NEVER accessible in derived class.

SECTION C — LONG ANSWER QUESTIONS (12 Marks Each)

Q. Explain different types of inheritance in C++ with diagrams and programs.

★ PYQ: ★★★★★ 2014,2017,2018,2021,2022,2024

Introduction:

Inheritance allows derived class to acquire properties of base class. Promotes code reusability.

Syntax:

```
class Derived : access_specifier Base { ... };
```

1. Single:

One base → One derived. [A → B]

2. Multiple:

Two+ bases → One derived. [A,B → C]

3. Multilevel:

Chain. [A → B → C]

4. Hierarchical:

One base → Many derived. [A → B,C,D]

5. Hybrid:

Mix of types. Can cause Diamond Problem.

Diamond Problem & Solution:

D inherits B and C which both inherit A → 2 copies of A. Solution: virtual base class: class B:virtual public A{};

```
// Single:
class Animal{public: void eat(){cout<<"Eat";}};
class Dog:public Animal{public: void bark(){cout<<"Bark";}};
// Multiple:
class C:public A,public B{};
// Multilevel:
class Animal{}; class Dog:public Animal{}; class Poodle:public Dog{};
// Virtual base (solves diamond):
class B:virtual public A{};
class C:virtual public A{};
class D:public B,public C{};
```

Conclusion: Inheritance is the most powerful OOP feature. Use virtual base class to resolve diamond problem.

Q. Discuss virtual functions and dynamic binding with C++ program.

★ PYQ: ★★★ 2018,2021,2022

Introduction:

Virtual functions enable runtime polymorphism — calling correct derived function via base pointer.

Rules:

Declared with 'virtual'. Overridden in derived class. Called through base pointer. Cannot be static.

Dynamic Binding:

Function call resolved at RUNTIME based on actual object type. Uses vtable internally.

Virtual Destructor:

Always declare destructor as virtual when using inheritance to prevent memory leaks.

```
#include
using namespace std;
class Shape{
public:
    virtual void area(){cout<<"Shape\n";}
    virtual ~Shape()
{}
};
class Circle:public Shape{
public: void area(){cout<<"Circle: pi*r*r\n";}
};
class Rect:public Shape{
public: void area(){cout<<"Rect: l*b\n";}
};
int main(){
    Shape *p;
    Circle c; Rect r;
    p=&c; p->area(); // Circle (dynamic binding)
    p=&r; p->area(); // Rect (dynamic binding)
    return 0;
}
```

Conclusion: Virtual functions + base pointers = runtime polymorphism. Always use virtual destructor in inheritance.

UNIT 5 — ADVANCED TOPICS — TEMPLATES, EXCEPTION HANDLING, FILE HANDLING

Multiple Inheritance · Virtual Base Classes · Pointers to Members · Function Templates · Class Templates · Exception Handling · File Handling

SECTION A — MCQ QUESTIONS (2 Marks Each)

MCQ 1. Generic type in a template function: [2018]

- (a) Must be T
- (b) Can be T ✓
- (c) Cannot be T
- (d) Only T for built-in

Explanation: Generic type CAN be named anything (T, U, TYPE) — 'T' is just a popular convention.

MCQ 2. Which is used to implement C++ interfaces? [2019]

- (a) Absolute variables
- (b) Abstract classes ✓
- (c) Constant variables
- (d) None

Explanation: C++ has no 'interface' keyword. Interfaces use abstract classes with all pure virtual functions.

MCQ 3. Exception handling keywords in C++: [2017]

- (a) try and error
- (b) try, catch, throw ✓
- (c) catch and handle
- (d) error and handler

Explanation: try: risky code. throw: signal error. catch: handle error. These 3 form C++ exception handling.

MCQ 4. Namespace is used to: [2019]

- (a) Group classes/objects/functions ✓
- (b) Mark program start
- (c) Separate classes
- (d) None

Explanation: Namespace groups related identifiers to avoid naming conflicts. 'using namespace std;' is standard.

MCQ 5. Scope resolution operator (::) is used to: [2019]

- (a) Create objects
- (b) Define members externally ✓
- (c) Declare classes
- (d) None

Explanation: :: defines class members outside class. Syntax: void ClassName::funcName(){} — required for external definition.

SECTION B — SHORT ANSWER QUESTIONS (4 Marks Each)

Q. What are templates? Explain need. Write syntax for function template.

★ PYQ: ★★★★★ 2014,2017,2018,2019

Templates enable generic programming — write once, use with any data type.

- template <class T> or template <typename T>
- Function template: template <class T> T funcName(T a, T b){ return ...; }

- Class template: `template <class T> class Stack{ T data[100]; };`
- Instantiate: `Stack<int> s;` — creates Stack for integers
- Advantage: Write once, use for int, float, char, any type.

Q. Discuss exception handling with try, catch, throw keywords.

★ PYQ: ★★★ 2014,2017,2018,2022

Exception handling manages runtime errors without crashing.

- try: Code that may throw exception.
- throw: Signals exception. throw expression;
- catch: Catches and handles exception. `catch(type var){ }`
- catch(...): Catches ALL exception types.
- Stack unwinding: Destructors called when exception thrown.

Q. What are types of file access? Explain file stream classes.

★ PYQ: ★★★ 2017,2021,2022

File I/O in C++ uses stream classes for reading and writing.

- Sequential Access: Read/write from start to end. For text files.
- Random Access: Jump to any position. For binary/database files.
- ifstream: Input (reading) | ofstream: Output (writing) | fstream: Both
- Key functions: `seekg()/seekp()` move pointer | `tellg()/tellp()` return position
- Always call `close()` after file operations!

SECTION C — LONG ANSWER QUESTIONS (12 Marks Each)

Q. What are templates? Explain function template and class template with examples.

★ PYQ: ★★★★★ 2014,2017,2018,2019

Introduction:

Templates enable GENERIC PROGRAMMING. Write once, use with any data type.

Function Template:

Creates a family of functions for different types.

Class Template:

Creates a family of classes for different types.

Advantages:

Write once for any type. Type-safe at compile time. No code redundancy. STL is built using templates.

```
// Function Template:
#include
using namespace std;
template
T findMin(T a,T b){ return (a
// Class Template:
template
class Stack{
    T data[100]; int top;
public:
    Stack(){top=-1;}
    void push(T val){data[++top]=val;}
    T pop(){return data[top--];}
    bool isEmpty(){return top==0;}
};

int main(){
    cout<< "Stack s;"
    s.push(10); s.push(20);
    cout<< "return 0;"
}
```

```
}
```

Conclusion: Templates are C++'s most powerful feature. The entire STL is built using templates.

Q. Define exception handling. Write program with try, catch, throw using constructor and destructor.

★ PYQ: ★★★ 2014,2017,2018,2022

Introduction:

Exception handling separates error code from normal code.

Flow:

try: normal code. throw: signal error. catch: handle error. Destructors called on stack unwinding.

Multiple catch:

Use different catch blocks for different exception types. catch(...) catches all.

```
#include
using namespace std;
class BankAccount{
    float balance;
public:
    BankAccount(float b){
        if(b<0) throw "Negative balance!";
        balance=b;
        cout<<"Account opened: "<<    }
    void withdraw(float amt){
        if(amt>balance) throw amt;
        balance-=amt;
        cout<<"Withdrawn: "<<    }
    ~BankAccount(){
        cout<<"Account closed. Balance:"<<    }
};
int main(){
    try{
        BankAccount acc(1000);
        acc.withdraw(500);
        acc.withdraw(800); // Throws!
    }
    catch(const char* msg){cout<<"Error: "<<    catch(float amt){cout<<"Insufficient! Need:"<<    return 0;
}
```

Conclusion: Exception handling with constructors/destructors ensures proper cleanup. Use multiple catch blocks.

UNIT ★ — MOST REPEATED VVI QUESTIONS — PYQ ANALYSIS 2014–2024

Rank	Question Topic	Unit	Times	PYQ Years
★★★★★	Constructor & Destructor (all types + programs)	U2	8+	EVERY YEAR 2014–2024
★★★★★	Types of Inheritance with diagrams & programs	U4	7	2014,2017,2018,2021,2022,2024
★★★★★	OOP Features/Paradigm (all features)	U1	7	EVERY YEAR 2014–2024
★★★★	Virtual Functions & Dynamic Binding	U4	6	2017,2018,2019,2021,2022,2024
★★★★	Operator Overloading + Type Conversion	U3	6	2014,2017,2018,2019,2022,2024
★★★★	Templates (Function + Class)	U5	5	2014,2017,2018,2019,2024
★★★★	Exception Handling with program	U5	5	2014,2017,2018,2022,2024
★★★	Abstract Class + Pure Virtual Function	U4	5	2017,2018,2019,2022,2024
★★★	Friend Function Program	U2	5	2014,2019,2021,2022,2024
★★★	File Handling Operations	U5	4	2017,2021,2022,2024
★★	Copy Constructor	U3	4	2017,2019,2021,2024
★★	Class vs Structure	U1	3	2018,2020,2022
★★	Static Data Members	U2	3	2019,2020,2022
★	Inline Functions	U2	3	2017,2020,2021
★	this Pointer	U2	2	2020,2022

UNIT 1 — LAST MINUTE CHEAT SHEET — EXAM READY REFERENCE

KEY ONE-LINER DEFINITIONS

Class:	Blueprint/template for creating objects.
Object:	Instance of a class with its own data and methods.
Encapsulation:	Binding data+methods in one unit. Provides data security.
Abstraction:	Hiding complexity; showing only what is needed.
Polymorphism:	Same name, different behavior in different contexts.
Inheritance:	Derived class acquires properties of base class.
Constructor:	Auto-called when object created. Same name as class, no return type.
Destructor:	Auto-called when object destroyed. ~ClassName(), no params.
Virtual Function:	Declared with 'virtual'; enables runtime polymorphism.
Abstract Class:	Class with at least one pure virtual function (=0). Cannot instantiate.
Template:	Generic programming. Works with any data type.
Exception:	Runtime error handled using try/catch/throw.
Friend Function:	NOT a member but can access private/protected data of class.
Static Member:	Shared across ALL objects. Only ONE copy for entire class.
this Pointer:	Pointer to current object inside non-static member functions.
Dynamic Binding:	Function call resolved at runtime via virtual functions (late binding).
Virtual Base Class:	Solves diamond problem in multiple inheritance. One copy of base.
Data Hiding:	Private members cannot be accessed from outside class.

MEMORY TRICKS

Topic	Memory Trick
5 OOP Features	'EAIPM' — Encapsulation, Abstraction, Inheritance, Polymorphism, Message passing
5 Inheritance Types	'SM-MHH' — Single, Multiple, Multilevel, Hierarchical, Hybrid
Cannot Overload (5)	:: (scope), . (dot), .* (member ptr), ?: (ternary), sizeof
Exception Keywords	'TTC' — Try (risky), Throw (signal), Catch (handle)
File Classes	'IOF' — ifstream (read), ofstream (write), fstream (both)
Constructor Types	'DPC' — Default, Parameterized, Copy
Access Specifiers	'PPP' — Private (class default), Protected (derived), Public (all)

COMMON EXAM MISTAKES TO AVOID

1. Destructor: Forgetting ~ symbol before class name.
2. Copy Constructor: Forgetting & (reference) in parameter — causes infinite recursion!
3. Pure Virtual: Forgetting = 0 at end of declaration.
4. Static Member: Not defining outside class: int ClassName::member = 0; is required.
5. Stream Operators: MUST be friend functions and MUST return stream reference.
6. Abstract Class: Cannot create objects — compilation error if you try!
7. Virtual Destructor: MUST use when inheriting to avoid memory leaks.

8. Derived class: Does NOT inherit constructors, destructors, and friend functions from base.
9. Operator Overloading: Cannot change precedence; cannot create new operators.
10. Template: Definition must be in header (.h) file, not just .cpp file.

Prepared for: AKU BCA 3rd Semester Students | Based on: Official Syllabus + PYQs 2014–2024 | Best of Luck! ✓